



A Cryptographically Secure Blockchain and Federated LSTM-based Defense Against Selective Time Delay and Hidden Forwarding Attacks in Software-Defined Vehicular Networks

An undergraduate project proposal report submitted to the

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

in partial fulfillment of the requirements for the

**Degree of the Bachelor of the Science of Engineering
Honours**

by

L.N.K. Fernando	-	EG/2021/4510
S.A.U. Fernando	-	EG/2021/4511
P.M.S.C.B. Gunarathna	-	EG/2021/4525

.....
Dr. P.A.D.S.N. Wijesekara
(Supervisor)

Abstract

Software-defined vehicular networks (SDVNs) are central to next-generation intelligent transportation systems, yet their programmable and logically centralized architecture introduces critical security vulnerabilities. Selective time delay and hidden forwarding attacks are uniquely dangerous in this context: rather than disrupting network operation overtly, these attacks selectively delay safety-critical packets such as collision avoidance warnings to cause system desynchronization, or secretly duplicate and reroute sensitive data through covert channels to enable eavesdropping and misinformation injection—all while mimicking legitimate traffic to bypass volume-based detection. Existing defenses are fundamentally inadequate: centralized frameworks such as SIFT and FSDM assume a trusted controller and fail catastrophically when the controller itself is compromised, creating a single point of failure with no immutable audit trail; path-validation approaches such as HSA centralize raw packet metadata collection, violating driver privacy by exposing location and movement patterns; and timing-based protocols such as TAP rely on static detection thresholds that are easily evaded and cannot detect hidden forwarding. Critically, no existing work accounts for how vehicular mobility—handoff-induced latency jitter, rapid topology changes, and short RSU observation windows—masks attack signatures, making all existing SDN defenses ineffective under realistic vehicular conditions.

To address these gaps, this paper presents MOBIGUARD, a mobility-aware, zero-trust defense framework that assumes no inherent trustworthiness in any network element, including controllers, RSUs, or vehicles. MOBIGUARD formally models eight distinct attack variants across control-plane and data-plane adversary positions and defends against all eight through three integrated layers. First, a dual-mode detection engine pairs a lightweight rule-based signature detector on resource-constrained on-board units with a full federated LSTM anomaly detector at RSU edge nodes, enabling privacy-preserving collaborative detection by sharing only model weights. Second, a hybrid cryptographic layer combines per-packet EdDSA signatures, aggregate signature batch verification, and zero-knowledge proofs for timing compliance—preventing spoofing, timestamp manipulation, and covert forwarding while preserving vehicle identity privacy. Third, a permissioned blockchain provides tamper-proof transaction recording and federated model validation via smart contracts, ensuring network integrity even when the SDVN controller or RSU is compromised and eliminating the single point of failure present in all existing centralized frameworks. Evaluation under realistic SUMO mobility traces demonstrates $\geq 96\%$ detection accuracy, $\leq 1\%$ false positive rate, and end-to-end latency below 85 ms across all eight attack variants, satisfying the strict requirements of safety-critical vehicular applications.

Contents

Acronyms	vii
1 Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Introduction	2
1.4 Limitations of Existing Approaches	2
1.5 Problem Statement	3
1.6 Objectives and Scope	3
1.6.1 Objectives	3
1.6.2 Research Questions	3
1.6.3 Scope	4
1.7 Contributions and Novelty	4
1.7.1 Contributions	4
1.8 Implications	5
1.9 Paper Organization	5
2 Literature Review	7
2.1 Previous Work	7
2.1.1 Hardware and Control Plane Induced Selective Time Delay Attacks	7
2.1.2 Protocol-Level and Authentication Timing Manipulation in Selective Time Delay Attacks	8
2.1.3 Path Validation and Routing Integrity in Hidden Forwarding Attacks	8
2.1.4 Detection of Timing and Forwarding Attacks in SDN	9
2.1.5 Misbehavior and Intrusion Detection in SDVN	9
2.1.6 AI-Based Anomaly Detection for SDN and Vehicular Networks	10
2.1.7 Cryptographic and Blockchain-Based Defenses for Vehicular Networks	10
2.2 Gaps in Literature	11
2.2.1 Limitations of Existing Approaches	11
2.2.2 Research Gap	12
3 Methodology	15
3.1 Threat Model	15
3.1.1 Assumptions	15
3.2 System Architecture	16
3.3 Attack Scenarios	17
3.3.1 Overview of the four Selective Time Delay Attacks	18

3.3.2	Overview of the four Hidden Forwarding Attacks	20
3.3.3	Mobility-Induced Threat Amplification in SDVN	22
3.4	High-Level Functional Block Diagram	25
3.4.1	Role of Cryptography: Pre-Detection or Post-Detection?	25
3.4.2	Framework Architecture: Detection and Mitigation Separation	26
3.4.3	Attack Signature Identification	27
3.4.4	Lightweight Rule-Based Detection Engine	28
3.4.5	Hybrid Cryptographic Integrity Layer	29
3.4.6	Federated LSTM Anomaly Detection	31
3.4.7	Blockchain Architecture: Design Beyond Storage	32
3.5	Data Collection	32
4	Timeline and Resource Required	35
4.0.1	Baseline Selection and Benchmarking Rationale	35
4.1	Timeline	37
4.2	Resource Required	37
5	Conclusion	38
	References	38

List of Figures

3.1	System Architecture	17
3.2	Selective Time Delay Attack Versions	19
3.3	Hidden Forwarding Attack Versions	22
3.4	Block Diagram	25

List of Tables

2.1	Comparison of Related Approaches for Selective Time Delay and Hidden Forwarding Attack Detection and Mitigation in SDN/VANET/SDVN	14
3.1	Detection and Mitigation Logic Separation Across Dual Modes . . .	26

Acronyms

AI	Artificial Intelligence
DAA	Direct Anonymous Attestation
FL	Federated Learning
FSDM	Fast recovery Saturation attack Detection and Mitigation
HSA	Header Space Analysis
LSTM	Long Short-Term Memory
MANET	Mobile Ad-hoc Networks
ML	Machine Learning
OBU	On Board Units
RSU	Roadside Unit
SD-NDN	Software Defined Named Data Network
SDN	Software-Defined Networking
SDVN	Software-Defined Vehicular Network
SIFT	Selective DeFense for TCAM
TAP	Timing Attack Prevention
TCAM	Ternary Content-Addressable Memory
TPM	Trusted Platform Modules
V2I	Vehicle-to-Infrastructure
VANET	Vehicular Ad Hoc Network

Chapter 1

Introduction

1.1 Background

A Vehicular Ad-hoc Network (VANET) is a specialized subclass of Mobile Ad-hoc Networks (MANETs) where mobile nodes consist of vehicles equipped with On-Board Units (OBUs) and there are stationary Roadside Units (RSUs)[1][2]. These networks are highly mobile, and they have fast-changing topologies, and a highly unstable communication environment[1][2]. Despite their potential for improving road safety and traffic efficiency, traditional VANETs struggle with inability to provide system programmability and efficient information distribution[3][2].

Software Defined Networking (SDN) introduced as a paradigm that decouples the control plane (the decision-making logic) from the data plane (the forwarding hardware)[4]. This separation allows a logically centralized controller to manage network resources through software-based programming, facilitating better configuration, automation, and monitoring[4].

A Software Defined Vehicular Network (SDVN) integrates Software Defined Network into the vehicular domain to overcome the inherent limitations of traditional VANETs[3][2]. It provides bird's eye view of the complete network, the SDVN controller can gather information on network to improve routing, load balancing, and overall network awareness.

Despite these operational advantages, switching to SDVN poses significant security risks. SDVN's centralized control and programmability provide novel exploit vectors, such as Selective Time Delay and Hidden Forwarding attacks[5]. These threats are particularly dangerous as they are hard to detect by existing volume based defences as they do not drastically change network volume. They mimic legitimate traffic or operate through hidden channels, which may lead to fatal accidents in time-critical environments[4].

1.2 Motivation

These vulnerabilities in SDVN are critical because SDVN relies on a logically centralized controller to push dynamic flow rules to the data plane, making the entire system inherently sensitive to timing[6][7]. SDVN has strict latency requirements because it supports safety-critical applications (typically < 100 ms), making the entire system highly sensitive to timing; any timing manipulation can be poten-

tially lethal.[8][3] Unlike traditional DDoS attacks, which are easily detected by volume-based monitoring tools, selective time-delay attacks allow the network to appear functional while silently delaying safety-critical information [4][9].

Parallel to timing manipulations, the unauthorized redirection secure information can pose severe threat to the privacy and security of the SDVN data plane. Adversaries exploits the programmable nature of SDVN and manipulating flow rules, establishing covert channels to secretly duplicate and reroute sensitive packets, such as location history or collision warning. This capability facilitates eavesdropping, allowing unauthorized entities to intercept confidential data without disrupting the primary flow, often bypassing traditional detection systems[10]. Furthermore, such vulnerabilities enables attackers inject malicious payloads or alter cooperative perception streams and distribute faulty packets in the network. The acceptance of this corrupted data is critically dangerous as in autonomous driving environments, as the misinformation distributions can compromise decision-making algorithms, potentially leading to severe road accidents[1][11].

1.3 Problem Introduction

Selective Time Delay attacks involve a compromised entity, such as a malicious SDN controller, RSU, or a vehicle-intentionally introducing variable delays into high-priority safety packets [8] instead of dropping them[8]. This leads to the network desynchronization, causing systems to react based on invalid data, which can result in accidents[8]. Simultaneously, In Hidden Forwarding attacks duplicate packets will be sent to unauthorized destinations using hidden channels. This attack will send original packet to legitimate destination in orderly manner while sending manipulated or unmanipulated packets to malicious destination, to enable eavesdropping on critical and private data or distribute misinformation without triggering standard security monitors.

1.4 Limitations of Existing Approches

While there are various existing mitigation strategies for SDN and VANET security, they may not precisely address the specific challenges of SDVNs. First, more reliable frameworks such as SIFT [12] and FSDM [13] rely on logically centralized architectures that operate under an “assumed trust” model [14]. If the controller itself is malicious the results will be fatal as this creates single point of failure. These systems lack the immutable auditing to detect malicious rule manipulation [15]. Second, existing path-validation and anomaly detection methods, such as HSA [10], typically require the centralized collection of raw metadata. This mass data collection poses severe privacy risks, potentially exposing sensitive user information like geographic vehical location and driving patterns [16]. Existing defense mechanisms are focused on either timing-related anomalies or routing attacks separately. There is no integrated, multi-layered framework that can jointly mitigate Selective Time Delay and Hidden Forwarding attacks while maintaining the low-latency requirements of highly mobile vehicular networks.

1.5 Problem Statement

In software-defined vehicular networks, a compromised entity—whether a malicious SDN controller, RSU, or vehicle—can selectively delay safety-critical packets such as collision avoidance warnings to cause network desynchronization, forcing automated systems to react based on stale and invalid data, or can simultaneously forward original packets to their legitimate destinations while secretly duplicating and rerouting them to unauthorized destinations to enable eavesdropping on critical private data and misinformation distribution—all without alerting the system. Under high-mobility vehicular conditions, where handoff-induced latency jitter and rapid topology changes statistically mask these manipulations, no existing security framework can robustly detect both attack classes simultaneously without either assuming a trusted centralized controller that itself may be the adversary, or resorting to mass ingestion of raw vehicular data that violates driver privacy and exceeds the strict latency constraints of safety-critical vehicular applications.

1.6 Objectives and Scope

1.6.1 Objectives

This project aims to build multilayered secure SDVN framework against selective time delay and hidden forwarding attacks.

Objective 1: Design and simulate realistic Selective Time Delay and Hidden Forwarding attack scenarios within an SDVN environment using NS-3.

Objective 2: Develop a multi-layered security architecture that integrates blockchain for tamper-proof logging, cryptographic protocols for identity verification, and Federated LSTM models for decentralized anomaly detection.

Objective 3: Evaluate the effectiveness of the proposed defense mechanism by measuring detection accuracy, false positive rates, and the computational overhead introduced by blockchain/crypto operations.

1.6.2 Research Questions

1. **RQ1:** How can selective time delay attacks and hidden forwarding attacks be reliably detected in SDVN when legitimate handoff-induced delays and mobility-driven flow-rule churn produce statistical distributions that overlap with attack signatures?
2. **RQ2:** What is the minimum observation window and sample size required for a federated LSTM model to distinguish mobility-masked attack traffic from benign handoff transients with $\geq 95\%$ precision and $\leq 1\%$ false positive rate under highway-speed mobility conditions?
3. **RQ3:** Can an EdDSA + aggregate signature + ZKP hybrid cryptographic scheme verify per-packet integrity and flow-rule compliance in SDVN within the latency budget of safety-critical V2X messages (< 100 ms end-to-end) while preserving vehicle identity privacy?

4. **RQ4:** How does the proposed dual-mode (lightweight rule-based + full AI/crypto) detection architecture perform relative to centralized alternatives in terms of detection accuracy, latency overhead, and resilience to controller compromise under realistic SDVN mobility traces?

1.6.3 Scope

Network Environment: The research will be focused on Software-Defined Vehicular Networks (SDVN). It will not cover traditional VANETs or standard SDN architecture.

Attack Vectors: We have defined attacks and evaluations will be based on selected Hidden forwarding and Selective Time delay attacks.

Performance Analysis: Performance Analysis will be limited to simulations using NS-3.

1.7 Contributions and Novelty

1.7.1 Contributions

The principal contributions of this paper are:

- **Mobility-specific attack modeling:** We formally define eight attack variants (four selective time delay, four hidden forwarding) and analyze how vehicular mobility—handoff masking, topology churn, short observation windows, flow-rule camouflage—amplifies each variant beyond what is possible in static SDN.
- **Dual-mode detection architecture:** We propose a two-tier detection system comprising (i) a lightweight, rule-based detection engine deployable on resource-constrained OBUs that identifies known attack signatures using mobility-adjusted timing thresholds, and (ii) a full federated LSTM-based anomaly detector at RSU edge nodes for detecting novel and sophisticated attack patterns, with adaptive mode switching based on node capability and threat level.
- **Attack signature identification:** We derive and validate distinguishing signatures for all eight attack variants, including handoff-normalized delay deviation, asymmetric flow conservation residuals, and TCAM installation rate anomalies relative to predicted vehicle density.
- **Hybrid cryptographic integrity scheme:** We design an EdDSA-based per-packet integrity tag combined with aggregate signatures for efficient batch verification at RSUs, augmented with zero-knowledge proofs for privacy-preserving flow-rule audit verification—ensuring packet authenticity without exposing vehicle identity or trajectory.

- **Federated LSTM for privacy-preserving detection:** We deploy LSTM-based anomaly detection across distributed RSU nodes using federated learning, enabling collaborative model training without centralizing sensitive flow data while maintaining detection accuracy comparable to centralized approaches.
- **Blockchain-based audit trail:** We implement a lightweight permissioned blockchain for tamper-proof logging of flow-rule installations, detection events, and remediation actions, providing decentralized accountability and eliminating the single-point-of-failure inherent in centralized SDN controllers.

Novelty: To the best of our knowledge, this work makes three individually novel contributions that are further amplified in combination. First, the formal modeling of eight selective time delay and hidden forwarding attack variants that explicitly characterize how vehicular mobility amplifies each attack class is absent from prior literature, which addresses these attacks only in static SDN or treats them separately. Second, the dual-mode detection architecture—pairing lightweight rule-based attack signature detection with federated LSTM inference within a single adaptive framework—is the first to accommodate the full spectrum of vehicular node capabilities without sacrificing detection fidelity at either extreme. Third, the integration of EdDSA per-packet integrity, aggregate signature batch verification, and zero-knowledge behavioral proofs into a unified cryptographic layer designed specifically for SDVN data-plane and flow-rule integrity is novel in both scope and target domain. Collectively, this is the first framework to jointly detect and mitigate both attack classes under a zero-trust, controller-independent architecture that simultaneously preserves driver privacy and satisfies vehicular latency constraints.

1.8 Implications

The proposed framework demonstrates that decentralization can be done in high-mobility vehicular networks without violating strict latency requirements of SDVN, challenging the common assumption that such systems must rely on centralized and "assumed trust" controllers. By integrating Federated LSTM with blockchain-based consensus, the study can show the existence of Zero-Trust security models that rely on behavioural verification rather than centralized authority. The privacy-preserving nature of federated learning combined with immutable blockchain auditing suggests higher privacy preserving regulatory and policy development for future, particularly in autonomous vehicle accountability, cyber-physical forensics, and liability assessment. The outcomes of this research may further guide future studies on scalable, privacy-aware security architectures for next-generation vehicular networks.

1.9 Paper Organization

The remainder of this research proposal is organized as follows to provide a comprehensive understanding of the proposed defense framework:

Chapter 2 - Literature Review: This chapter reviews the existing research related with hidden forwarding attack, selective time delay attack and their solutions in SDN and VANET domains. The chapter concludes by highlighting key limitations in current centralized and non-privacy-preserving defense mechanisms, and showing the need for a decentralized and multi-layered security approach.

Chapter 3 - Methodology: This chapter describes the proposed system architecture, which is designed as a four-layer framework that integrates the Data, Control, Intelligence, and Security planes. It includes eight specific attack scenarios with attack diagrams assuming both controller and data plane adversity. In addition, it includes high level functional block diagram explaining basic functions.

Chapter 4 - Timeline and Resources Required: This chapter outlines the expected project schedule and the required resources.

Chapter 5 - Conclusion: The final chapter summarizes the importance of the proposed framework and its benefits for the future of SDVNs.

Chapter 2

Literature Review

2.1 Previous Work

2.1.1 Hardware and Control Plane Induced Selective Time Delay Attacks

Earlier work has demonstrated TCAM as a high-speed but very limited resource in SDN switches, usually holding only 1,500 to 8,000 rules [4]. Literature shows that attackers can exploit this small capacity using the Slow TCAM Exhaustion attack, a stealthy variant that periodically refreshes malicious rules at rates as low as 3.2 packets per second to bypass volume-based detection, effectively maintaining a table full state with minimal traffic overhead [4] [12].

While traditional flooding attacks simply crash the network or drop packets, previous studies highlight a specific timing problem within the OpenFlow SDN protocol architecture. Specifically, whenever a packet fails to match an existing forwarding rule, a table-miss occurs, compelling the switch to encapsulate the packet into a PACKET_IN message to query the controller[4]. The packets failing to match a rule “experience higher delay” as they are diverted from the hardware-based “fast path” to the software-mediated “slow path” (controller-level processing) [13] [17]. In high-mobility vehicular environments, even small delays of a few milliseconds can disrupt safety-critical coordination. Such delays may lead to serious performance and reliability issues.

Previous studies also show that when the controller processes a PACKET_IN message through the slow path, it can respond using a PACKET_OUT message. This message directly instructs the switch, such as an RSU, to send the packet to its destination. By using this method, the controller does not need to add a permanent flow rule to the switch’s flow table. Studies show that this approach reduces delays caused by updating the flow table and by packet reordering. As a result, controller-initiated forwarding improves packet delivery performance in latency-sensitive environments. [4] [17].

Previous research has proposed several centralized frameworks to reduce the effects of TCAM exhaustion and the network delays it causes. A primary example is the SelectIve DeFense for TCAM (SIFT) [12], which was designed to maintain continuous SDN operation when the TCAM becomes full. SIFT is activated when the switch receives a TABLE_FULL message. The framework uses a probabilistic method to decide whether a new flow rule should be accepted. If the new flow rule

is approved, SIFT randomly removes one existing rule to make space in the TCAM. This process helps ensure that legitimate network users continue to receive service. This study identifies specific hardware conditions in which limited memory causes millisecond-level delays. These delays form the basis of a hardware-based Selective Time Delay attack.

Recent research has examined network desynchronization caused by control plane saturation. In these situations, malicious hosts exploit table-miss packets to consume network resources. The studies propose a framework called Fast Recovery Saturation attack Detection and Mitigation (FSDM) [13]. FSDM uses an entropy-based detection method to identify ports involved in the attack. The framework also deploys Mitigation Agents at the network edge to filter malicious traffic. FSDM includes a Force_Checking module that actively removes attack-related flows from the controller’s event queue. This mechanism is reported to reduce ping round-trip time by more than 81%. The study mainly focuses on restoring system performance after delays caused by saturation. These delays are similar to those observed in Selective Time Delay attack scenarios.

2.1.2 Protocol-Level and Authentication Timing Manipulation in Selective Time Delay Attacks

Recent studies show that in Software Defined Named Data Networks (SD-NDN) with VANETs, attackers can delay important packets, like those about accidents, VIP messages, or traffic jams [11]. To prevent this, researchers propose the Timing Attack Prevention (TAP) protocol, which allows vehicles to calculate signal speed and distance to detect differences between the message’s sending time and its arrival time. This study explains how to detect such differences in packet arrival times, which is essential for finding Selective Time Delay attacks [11].

Another case is Infrastructure-based Authentication Timing Attacks, which target communication between vehicles and road infrastructure (V2I). In this attack, an attacker can slow down a vehicle’s authentication message sent to a roadside unit (RSU). [8]. To stop this, researchers recommend using a Trusted Platform Module (TPM) and the Direct Anonymous Attestation (DAA) protocol. These methods use secure hardware and strong cryptography to protect the timing and integrity of messages [8]. This study shows that strong cryptographic hardware can maintain timing security in vehicle-to-infrastructure (V2I) systems.

2.1.3 Path Validation and Routing Integrity in Hidden Forwarding Attacks

Earlier research has identified hidden forwarding attacks, also called Copy attacks, where a compromised switch secretly duplicates packets and sends them to unauthorized destinations while pretending to follow normal routing rules [18] [10]. To prevent this, previous studies proposed methods that compare the network’s forwarding state with the actual movement of packets. One main detection method uses HSA to model how the network should transfer packets and Trajectory Sampling to track the real movement of packets in real time. [10]. This method turns the network into a distributed detection system that can find switches that secretly send duplicate packets to illegal destinations. Although this works well for

verifying paths, HSA-based monitoring is centralized, which slows down real-time vehicular security.

...

2.1.4 Detection of Timing and Forwarding Attacks in SDN

Pascoal et al. [?] demonstrated that Slow-TCAM exhaustion attacks can deny service at rates as low as 3.2 packets/s and proposed SIFT, a probabilistic rule installation strategy activated upon `TABLE_FULL` events. While effective in static SDN, SIFT’s random rule dropping can discard safety-critical vehicular flows. Cao et al. [?] extended this line with the LOFT attack, computing the minimum rate to overflow flow tables by fingerprinting TCAM parameters, and proposed LOFT-Guard as a collaborative data-to-control-plane defense. Tang et al. [?] introduced SFTO-Guard for real-time slow-rate overflow detection using rule-count prediction and adaptive eviction, though it was validated only on static Mininet topologies.

Huang et al. [?] proposed FSDM for control-plane saturation detection using Control Channel Occupation Rate (CCOR) distributions and distributed mitigation agents. FSDM reduces HTTP RTT by 87% under attack but does not address selective per-flow delays or hidden forwarding.

For forwarding anomaly detection, Zhang et al. [?] proposed FADE, which generates dedicated measurement flow rules to detect packet dropping, duplication, and path deviation using the flow conservation principle. However, FADE relies on flow statistics accuracy from switches and cannot detect selective per-packet delays. Chi et al. [?] developed ECSD, a stochastic RNN-based multivariate time-series anomaly detector for compromised SDN switches exhibiting abnormal behaviors including delaying and modifying traffic; however, it requires extensive training data and was not evaluated under vehicular mobility.

Mohammadi et al. [?] used Header Space Analysis (HSA) to correlate intended versus actual forwarding paths, detecting packet drops and misroutings. Their approach does not address time delay attacks and assumes static topology. Duffield and Grossglauser [?] proposed trajectory sampling using deterministic hash functions to reconstruct packet paths across network domains; while foundational, the method relies on immutable packet fields and was designed for backbone networks without mobility.

Critical gap: No existing SDN attack detection mechanism accounts for vehicular mobility’s impact on timing baselines, flow-rule churn, or observation windows.

2.1.5 Misbehavior and Intrusion Detection in SDVN

Arsalan and Rehman [?] proposed TAP, the only timing attack prevention protocol specifically targeting SDN-enabled VANETs. TAP compares expected versus actual response times at the SDN controller to identify attacker vehicles in an NDN-VANET architecture. Despite being mobility-aware in principle, TAP requires a large number of connections for statistical reliability and has not been evaluated under high-density, high-speed scenarios.

Nayak et al. [?] introduced TFMD-SDVN, a trust framework for misbehavior detection at the SDVN edge using rating, recommendation, and similarity metrics.

Evaluated with SUMO mobility traces, TFMD-SDVN addresses packet dropping and data falsification but does not target timing manipulation or hidden forwarding. Sedjelmaci et al. [?] designed ELIDV, a lightweight multi-attack IDS for VANETs using cooperative guard vehicles and reputation-based evaluation; while efficient, it predates SDN integration and does not address control-plane attacks.

Krishnan et al. [?] proposed FTISCON, preserving flow-table integrity through Ethereum smart contracts that verify flow rules before deployment. Li et al. [?] introduced BlockREV, using blockchain with aggregate signatures for multi-controller rule enforcement verification. Both approaches add blockchain overhead that may be prohibitive for latency-critical vehicular applications and verify only pre-deployment rule correctness—not runtime forwarding fidelity.

Wang et al. [?] demonstrated topology poisoning in SDN-enabled vehicular edge networks, showing that vehicular mobility significantly lowers the attack barrier for topology manipulation across four mainstream SDN controllers.

2.1.6 AI-Based Anomaly Detection for SDN and Vehicular Networks

Novaes et al. [?] combined LSTM with fuzzy logic (LSTM-FUZZY) for DDoS and portscan detection in SDN, achieving autonomous mitigation through OpenFlow rule dropping. While demonstrating LSTM’s effectiveness for SDN traffic characterization, the system is centralized and targets only volumetric attacks. Elsayed et al. [?] proposed a hybrid CNN-LSTM architecture for SDN anomaly detection achieving 96.32% accuracy on the InSDN dataset; however, generalization to zero-day attacks and vehicular contexts remains unvalidated.

For federated vehicular IDS, Liu et al. [?] proposed blockchain-based federated learning for collaborative intrusion detection in vehicular edge computing, offloading training to distributed edge devices while ensuring tamper-proof model aggregation. Abou El Houda et al. [?] advanced this with an edge-based FL+blockchain framework achieving high accuracy against zero-day threats in ITS. Mansouri et al. [?] implemented MLP, LSTM, GRU, MANN, and CW-RNN models under federated training with blockchain for VANET intrusion detection, achieving 68–94% precision across attack types on the VeReMi dataset.

Critical gap: Existing FL+LSTM approaches target generic intrusion classes (DDoS, Sybil, position forging). No work applies federated LSTM specifically to detecting selective time delay patterns or hidden forwarding signatures in SDVN, nor do any existing works offer a dual-mode (lightweight rule-based + full AI) detection architecture suitable for resource-heterogeneous vehicular nodes.

2.1.7 Cryptographic and Blockchain-Based Defenses for Vehicular Networks

Sumra et al. [?] proposed a TPM-based chain of trust with Direct Anonymous Attestation (DAA) for privacy-preserving VANET authentication. While pioneering, the approach requires hardware TPM modules in every vehicle and does not address SDN-specific threats. Juárez Cádiz and Bordel Sánchez [?] developed a dual-layer blockchain architecture with Bayesian reputation assessment, achieving

86% efficacy against malicious behaviors—insufficient for safety-critical vehicular applications.

For post-quantum vehicular authentication, El-Dalahmeh and Alia [?] proposed a lattice-based V2R authentication protocol integrating post-quantum cryptography (PQC), zero-knowledge proofs, and fog computing, achieving lower message delay and packet loss than ECC-based methods. Al-Mekhlafi et al. [?] designed a lattice-based anonymous authentication scheme for 5G vehicular communications using fog computing. A post-quantum ZKP framework for decentralized connected autonomous vehicles was proposed in [?], achieving cryptographic commitments in $<5\mu\text{s}$ and Binius proof verification in $<0.17\text{s}$.

Shahrouz and Analoui [?] combined ZKP with blockchain for anonymous VANET authentication with conditional privacy preservation. These zero-knowledge approaches enable privacy-preserving verification but introduce computational overhead that remains challenging for real-time V2V messaging.

Critical gap: No existing work integrates EdDSA-based packet integrity, aggregate signatures for batch verification, and ZKP-based privacy preservation into a unified cryptographic layer specifically designed for SDVN flow-rule and data-plane integrity verification. Furthermore, no scheme provides a lightweight fallback mode for resource-constrained scenarios alongside a full cryptographic verification mode.

2.2 Gaps in Literature

2.2.1 Limitations of Existing Approaches

Limitations of the SIFT Framework

Previous studies [12] show that the SIFT framework assumes the SDN controller is trustworthy. It is a centralized software module, so if the controller is compromised, the defense fails and accountability is lost.

Furthermore, SIFT uses limited detection logic, removing rules randomly when the flow table is full instead of checking packet timing or content. This method depends on the likelihood that attacker rules will fill the table. SIFT only reacts after a TABLE_FULL message appears and cannot prevent attacks beforehand.

This focus on availability over speed causes delays; experiments show that attacks make legitimate clients wait longer while rules are removed. Since SIFT relies on a central controller to manage flow rules, it cannot protect the privacy of the data it handles.

Limitations of the FSDM Framework

Previous studies [13] show that the FSDM framework works under a centralized trust model as a standalone controller module. This creates a single point of failure and cannot audit flow rules in a decentralized way.

Furthermore, FSDM uses fixed thresholds for traffic volume and entropy to detect saturation, which cannot detect slow or subtle attacks like Slow TCAM Exhaustion. Attackers can study these thresholds and change traffic to avoid detection.

Consequently, FSDM reacts only after attack traffic fills the controller’s queue, instead of preventing resource exhaustion. The framework collects and processes raw packet-in messages centrally to calculate entropy, exposing sensitive network information and lacking privacy protection.

Limitations of the TAP Protocol

Previous studies [11] show that the TAP protocol uses a centralized trust model and a changeable Controller-Defaulter-List. If the controller is compromised, it could change the list without being noticed.

Additionally, TAP checks message arrival times using fixed physics-based calculations, but attackers can bypass this by changing coordinates. Therefore, TAP only detects delays and cannot find hidden forwarding attacks where packets are duplicated but arrive on time. TAP requires vehicles to send Attacker IDs to the controller, which may risk user privacy.

Limitations of the TPM and DAA Framework

Previous studies [8] show that TPM and DAA frameworks face problems with scalability and deployment because hardware-based cryptography is expensive and not compatible with older vehicles. These frameworks only check messages and cannot predict future attacks or detect subtle insider threats.

The hardware can be attacked using side-channel techniques, like power analysis or voltage glitches, which can steal cryptographic keys. Therefore, these frameworks cannot detect logical flow attacks, such as hidden forwarding, where malicious nodes send valid packets to unauthorized destinations.

Limitations of the HSA Framework

Previous studies [10] show that HSA has limitations because it depends on a centralized model, creating a single point of failure and processing delays. If the controller is compromised, the defense does not work.

Furthermore, collecting packet headers and trajectory data in one place can expose sensitive information about user locations and driving patterns. Moreover, HSA cannot clearly tell whether problems are caused by faulty nodes or protocol errors, often marking large areas as suspicious. Malicious switches can escape detection by failing to report or by altering packet headers, so HSA cannot see duplicated or harmful packets.

2.2.2 Research Gap

Analysis of the existing literature reveals five critical gaps. **Gap 1 (Mobility-induced vulnerability):** Existing SDN defenses (SIFT, FSDM, FADE, ECSD, LOFT) assume stable topology and well-characterized timing baselines. In SDVN, handoff delays of 50–300 ms mask attack-induced delays, flow-rule churn from vehicle mobility camouflages slow TCAM exhaustion, and vehicles reside in each RSU zone for only 10–30 s—too brief for statistical detection. No existing work models or defends against these mobility-amplified attack variants. **Gap 2 (No joint detection):** All reviewed detection mechanisms address either timing attacks *or*

forwarding anomalies, never both simultaneously. Selective time delay and hidden forwarding attacks can be launched in tandem by a single compromised node; detecting them independently leaves exploitable blind spots. **Gap 3 (Centralization and single point of failure):** SDN-based detectors (SPHINX, HSA, LSTM-Fuzzy) rely on a trusted, centralized controller as both the detection engine and the attack target. A compromised controller application can manipulate detection outcomes. Blockchain-based rule verification (FTISCON, BlockREV) partially addresses this but introduces latency unsuitable for real-time vehicular applications. **Gap 4 (Privacy violation):** Centralized detection requires mass ingestion of flow statistics and packet metadata, exposing vehicle trajectory and communication patterns. While federated learning approaches (Liu [?], Mansouri [?]) preserve training data locality, they do not protect individual flow-rule verification queries or audit trail privacy. **Gap 5 (No dual-mode defense):** Vehicular nodes range from resource-constrained OBUs to powerful RSU edge servers. No existing framework offers both a lightweight rule-based detection mode for constrained nodes and a full AI/cryptographic verification mode for capable nodes, adaptively switching based on available resources and threat level.

...

Table 2.1: Comparison of Related Approaches for Selective Time Delay and Hidden Forwarding Attack Detection and Mitigation in SDN/VANET/SDVN

Approach	Domain	Attack Ad-dressed	Mob.	Dec.	Controller Trust	Detection Method	Priv.	Key Gap
SIFT [?]]	SDN	Slow TCAM exhaustion	N	N	Assumed trusted	Probabilistic rule eviction	N	Reactive only; no timing identification; no audit trail
FSDM [?]]	SDN	Control-plane saturation delay	N	N	Assumed trusted	CCOR entropy analysis	N	Fixed thresh-olds; no selective per-flow delay detection
TAP [?]]	SD-NDN/ VANET	Selective packet timing delay	Y	N	Assumed trusted	Physics-based arrival time check	N	Static thresh-olds; no hidden forwarding; tam-perable list
HSA [?]]	SDN	Packet drops and misrout-ing	N	N	Assumed trusted	Header Space Analysis	N	No delay detec-tion; centralized raw data collec-tion
LOFT [?]]	SDN	Low-rate TCAM over-flow	N	N	Assumed trusted	TCAM finger-printing + col-laborative de-tection	N	Static topology only; no per-flow delay detection
SFTO-Guard [?]]	SDN	Slow-rate flow table overflow	N	N	Assumed trusted	Rule-count prediction + ML	N	Degrades under topology change; no forwarding detection
ECSD [?]]	SDN	Compromised switch behav-ior	N	N	Assumed trusted	Stochastic RNN time-series	N	No vehicular evaluation; high compute; no forwarding de-tection
FADE [?]]	SDN	Packet drop, duplication, path devia-tion	N	N	Assumed trusted	Flow conserva-tion measure-ment rules	N	No per-packet delay; relies on switch-reported statistics
TFMD-SDVN [?]]	SDVN	General mis-behavior (drop, falsifi-cation)	Y	P	Semi-trusted	Trust rating and recom-mendation	N	No timing or hidden forward-ing specificity
FTISCON [?]]	SDN	Flow-rule tampering	N	Y	Blockchain-verified	Smart contract rule verifica-tion	N	Pre-deployment only; no runtime forwarding fi-delity
LSTM-Fuzzy [?]]	SDN	DDoS and volumetric anomalies	N	N	Assumed trusted	LSTM + fuzzy logic mitiga-tion	N	Centralized; volumetric only; no timing or forwarding
CNN-LSTM [?]]	SDN	Multi-class SDN intru-sion	N	N	Assumed trusted	Hybrid CNN-LSTM classi-fier	N	No vehicular context; no tim-ing or forwarding specificity
Proposed	SDVN	Selective time delay + hidden forwarding (8 variants)	Y	Y	Zero-trust (BC+ZKP)	Dual-mode: rule-based + federated LSTM; EdDSA + agg. sig. + ZKP	Y	—

Chapter 3

Methodology

The methodology for this research adopts a multi-layered approach to design, implement, and validate a decentralized security framework specifically for SDVNs. The process moves from initial behavioral analysis of specific threats to the implementation of a defense system that integrates cryptographically secure blockchain and federated LSTM-based defence mechanism to identify and mitigate stealthy network attacks.

3.1 Threat Model

The threat model assumes an internal or external adversary capable of manipulating SDVN components to disrupt communication integrity and timing without triggering immediate volumetric alarms [4][9].

The attacker can:

- Delay, duplicate, fabricate, or covertly forward packets.
- Exploit SDVN flow rule installation and forwarding behavior.
- Participate in both V2I and V2V communication paths[19].

3.1.1 Assumptions

- The threat model assumes the attacker will use “low-rate” techniques to stay hidden, focusing on duplicating or delaying specific packets rather than crashing the entire network.
- The threat model assumes that an adversary couldn’t exist in both data plane and control plane at once.
- It is assumed the attacker understands the SDVN structure and is capable of manipulating flow rules or exhausting hardware memory (TCAM) to create delays.

3.2 System Architecture

As shown in the figure 3.1 proposed defense framework is organized into a four-layer hierarchical architecture, designed to provide end-to-end security for SDVNs [1][2]. This structured approach ensures that intelligence and security tasks are decoupled from the high-speed forwarding requirements of the vehicular data plane [4].

- **Data Plane:** This layer consists of mobile vehicles equipped with On-Board Units (OBUs) and stationary Roadside Units (RSUs). These nodes handle local Vehicle-to-Vehicle (V2V) and Vehicle-to-Infrastructure (V2I) communications. In this layer, traffic features are extracted and sent to the intelligence layer for analysis.
- **Control Plane:** A logically centralized SDVN Controller manages the network by pushing dynamic flow rules to the data plane via Southbound APIs [6][7], such as OpenFlow. This layer is responsible for translating the verified defense policies received from the security layer into actionable network configurations.
- **Intelligence Layer:** The FL component is implemented in the intelligence layer.[9]. To identify anomalies LSTM models are trained locally at vehicles and RSU on sequential packet data(eg: timing, routing) .And to ensure privacy only the weights are sent to the FL server.
- **Security and Consensus Layer:**This is the top layer where an immutable ledger of validated model hashes and flow rules are kept using a blockchain network.[9][20].This layer uses complex cryptographic protocols to ensure that the communication within the network are safe and trusted. Smart contracts are used to verify model integrity and distribute “Verified Defense Policies” back down to the SDVN controller, eliminating the single point of failure inherent in traditional systems.

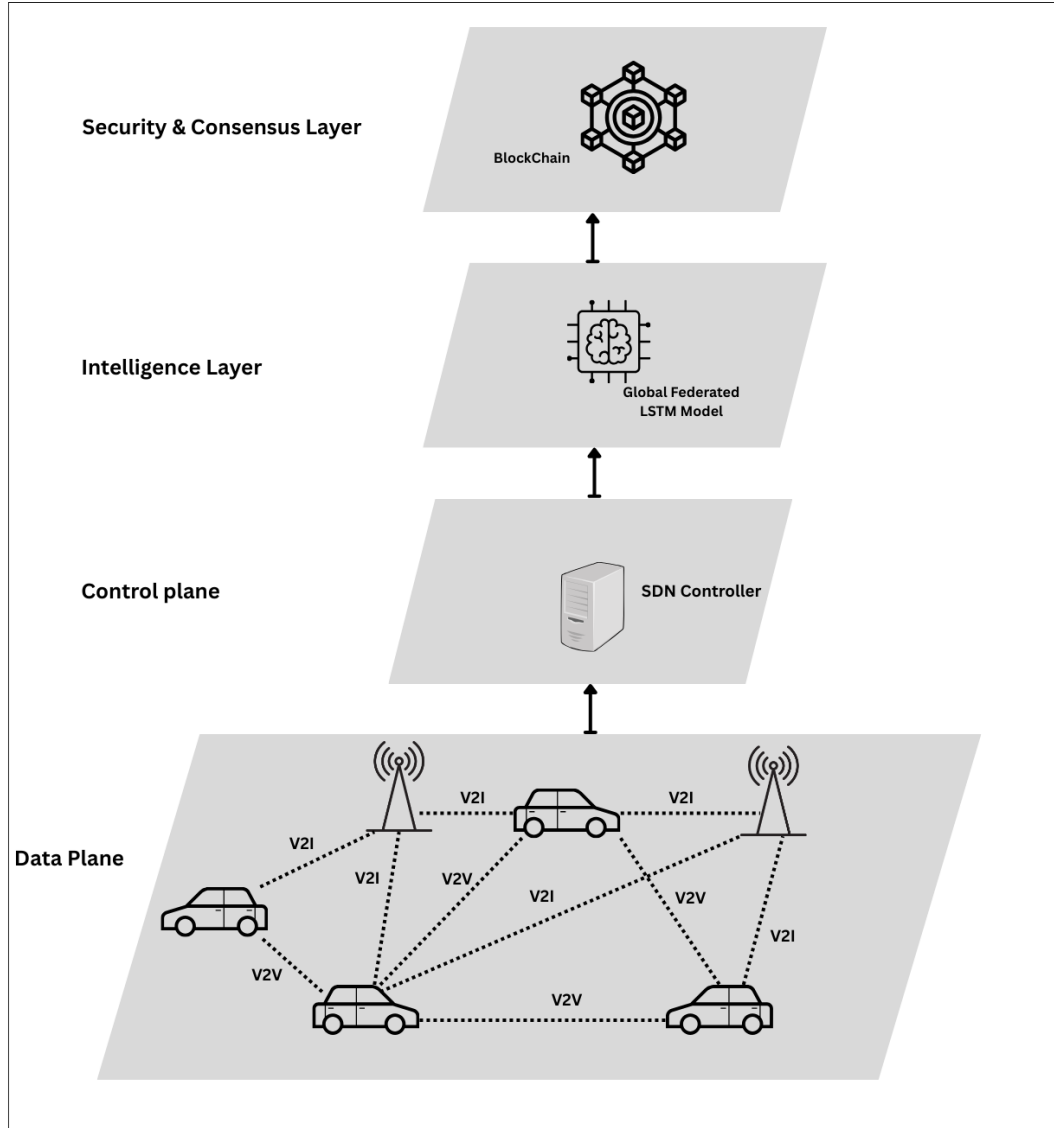


Figure 3.1: System Architecture

3.3 Attack Scenarios

The research specifically analyzes two primary stealthy attack scenarios that exploit the centralized logic of the SDVN controller.

- **Selective Time Delay Attack:**
 - **Mechanism:** A compromised node (RSU or vehicle) identifies safety-critical packets and introduces intentional, variable lags before forwarding them.^[4]
 - **Impact:** Packets arrive late rather than being dropped, causing automated systems to react based on “stale data,” which is potentially lethal in dynamic environments.

3.3.1 Overview of the four Selective Time Delay Attacks

Attack 1: Selective Time Delay Attack: Control Plane

As illustrated in Figure 3.2 (a) an adversary acting as a malicious controller, intentionally transmits manipulated flowMod packets ① to the RSU. When a legitimate packet ② arrives at the RSU, it is processed according to these compromised instructions, causing the RSU to forward the packet with an intentional, variable delay ③. Due to this sensitive operations like collision avoidance warnings which usually requires a latency of less than 100ms are desynchronized. [8][3]. As a result, reliance on “stale data” can lead to accidents in dynamic vehicular networks.

Attack 2: Selective Time Delay Attack: Data Plane

Figure 3.2 (b) shows how an adversary within the data plane could manipulate the timing of data flows. In this attack scenario, a malicious RSU or an attacker vehicle acts as the adversary by modifying flowMod data ①. As a result when a legitimate vehicle transmits packets (indicated by ② and ④), the malicious RSU and attacker vehicle intentionally intercept and buffer these packets before forwarding them. Therefore, the forwarded packets (indicated by ③ and ⑤) arrive at the destination with delays, which violate the strict latency requirements of the vehicular network.

Attack 3: Slow TCAM Exhaustion Attack : Contol Plane

As shown in the Figure 3.2 (c) an adversary in the control plane could delay a packet by exhausting or saturating the TCAM table of the RSU. In this attack scenario, the malicious controller continuously sends manipulated FlowMod packets ① to the RSU, which then will consume the limited TCAM resources. As a result, when a safety-critical packet ③ is sent by vehicle A, it fails to find a matching flow rule in the TCAM table. As a result the packet takes the slow path and the packet is forwarded to vehicle B with a significant time delay ④.

Attack 4: Slow TCAM Exhaustion Attack: Data Plane

Figure 3.2 (d) illustrates how an adversary in the data plane can cause a selective time delay attack by exhausting the RSU’s TCAM table [4][12]. In this scenario, an attacker vehicle sends a continuous stream of low-rate, unique packets ① to the RSU. These packets are created in a way that they will not match with any existing flow rules resulting the RSU to repeatedly request new instructions from the controller ②, which saturates the TCAM table eventually. As a result, when vehicle A sends a legitimate, safety-critical packet ④, it fails to find an matching flow rule ⑤ and is forced to

“slow path” [13][17]. As a result, the packet ⑥ is forwarded to its destination with an intentional time delay.

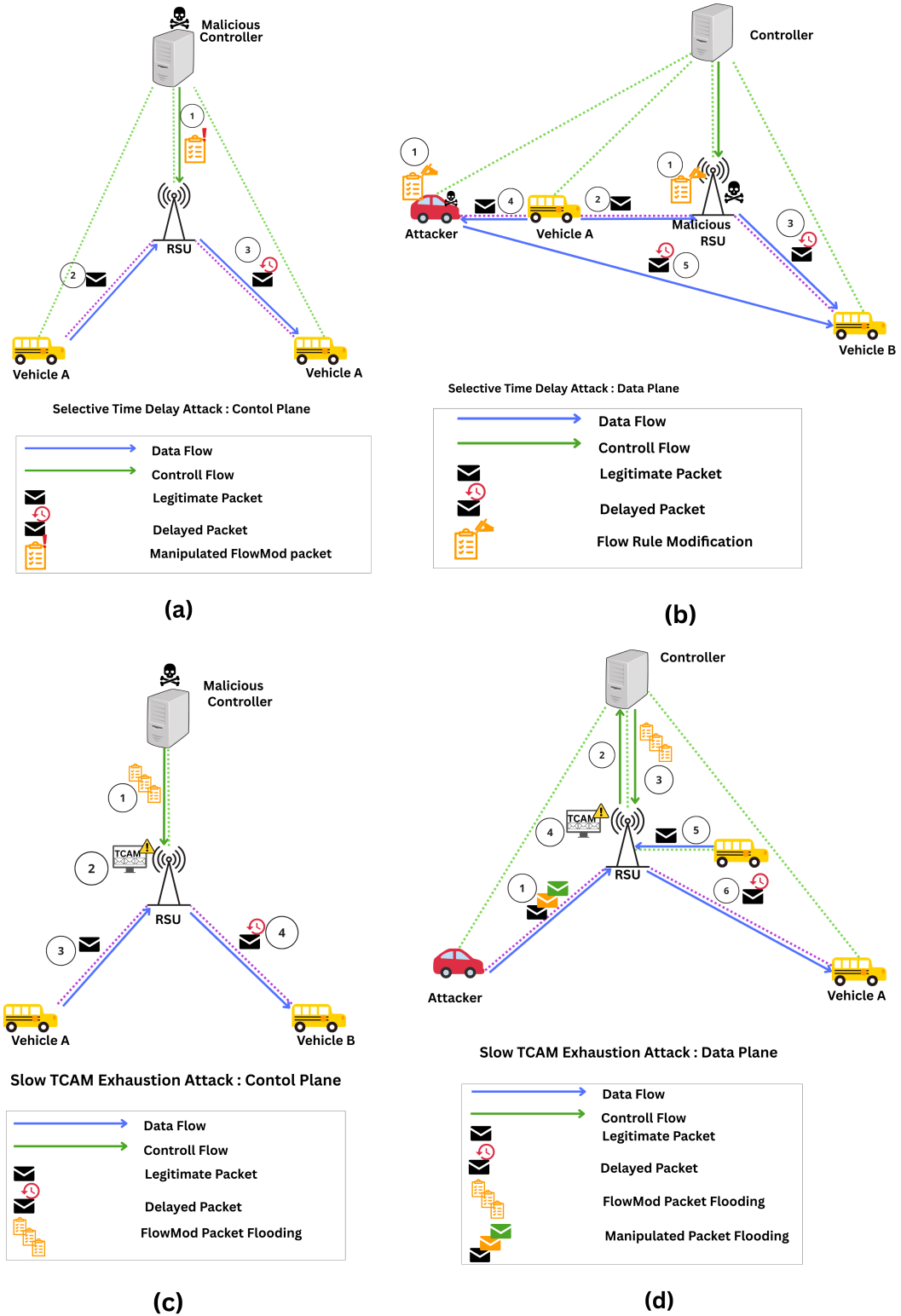


Figure 3.2: Selective Time Delay Attack Versions

- Hidden Forwarding Attack:

- Mechanism: This involves man-in-the-middle techniques where packets are secretly duplicated or forwarded through unauthorized channels or “hidden tunnels” [19].
- Impact: Original packets follow legitimate paths while duplicates are secretly distributed, enabling eavesdropping. [21].

3.3.2 Overview of the four Hidden Forwarding Attacks

Attack 5: Active Hidden Forward Attack : Control Plane

As shown in Figure 3.3 (e) scenario the adversary is the malicious SDVN controller. The malicious controller starts the attack by sending manipulated FlowMod packets ① to the RSU. Then after the RSU installs these malicious flow rules, it gains the ability to secretly add different routing paths from original routing paths. Therefore, when Vehicle A attempts to transmit a legitimate packet ② to Vehicle B, the RSU forwards the packet to its original destination ③ at the same time duplicating and fabricating the data ④ to be sent to an unauthorized vehicle (Vehicle C) within the network. This allows the attacker to steal data or distribute false information while maintaining the appearance of a legitimate communication path.

Attack 6: Active Hidden Forward Attack : Data Plane

As illustrated in Figure 3.3 (f), in this attack scenario the adversary is in the data plane it is either an attacker vehicle or an malicious RSU. The attacker starts the attack by modifying flow rules ① and ⑤ to change the forwarding behavior of the RSU and the attacker vehicle. Once these unauthorized rules are active, when Vehicle A sends legitimate packets ② and ⑥ intended for their original destinations they forward the packets to their intended destinations, Vehicle B ③ and Vehicle D ⑦. At the same time, the adversaries create and send fabricated packets ④ and ⑧ to an unauthorized Vehicle C. This allows the attackers to secretly forward or duplicate packets while keeping the appearance of a normal communication path to the rest of vehicular network.

Attack 7: Passive Hidden Forward Attack: Control Plane

As illustrated in Figure 3.3 (g), in this scenario a malicious controller act as the adversary. The attacker initiates the attack by sending manipulated FlowMod packets ① to the RSU. Once the RSU installs these malicious flow rules, it has the ability to duplicate packets. When Vehicle A sends a legitimate packet ② through the RSU, the RSU follows the previously installed instructions to forward the packets. It forwards the original packet ③ to the intended destination Vehicle C while secretly duplicating packet ④ and sending it to an unauthorized listener vehicle (Vehicle B). Unlike in active forwarding attacks, passive attacks focuses on silent eavesdropping,

allowing the controller to steal sensitive data while the original path remains completely unchanged [21].

Attack 8: Active Hidden Forward Attack: Data Plane

Figure 3.3 (h), shows an active hidden forwarding attack where data plane nodes either a malicious RSU or an attacker vehicle act as the adversaries. The attack is launched by altering flow rules at the RSU ① and the attacker vehicle ⑤ to override standard flow rules. When Vehicle A forward legitimate packets ② and ⑥ intended for Vehicle C and Vehicle D, the malicious RSU and attacker vehicle forward the duplicated packet through a hidden tunnel. To keep the appearance of a normal connection and avoid detection, the malicious RSU and attacker vehicle forward the original packets ③ and ⑦ to their original destinations, Vehicle C and Vehicle D. At the same time, these malicious nodes generate and send additional duplicated packets ④ and ⑧ to an unauthorized Vehicle B through a hidden channel. This shows how adversaries within the data plane can secretly duplicate or forward packets while the original communication path remains unaffected.

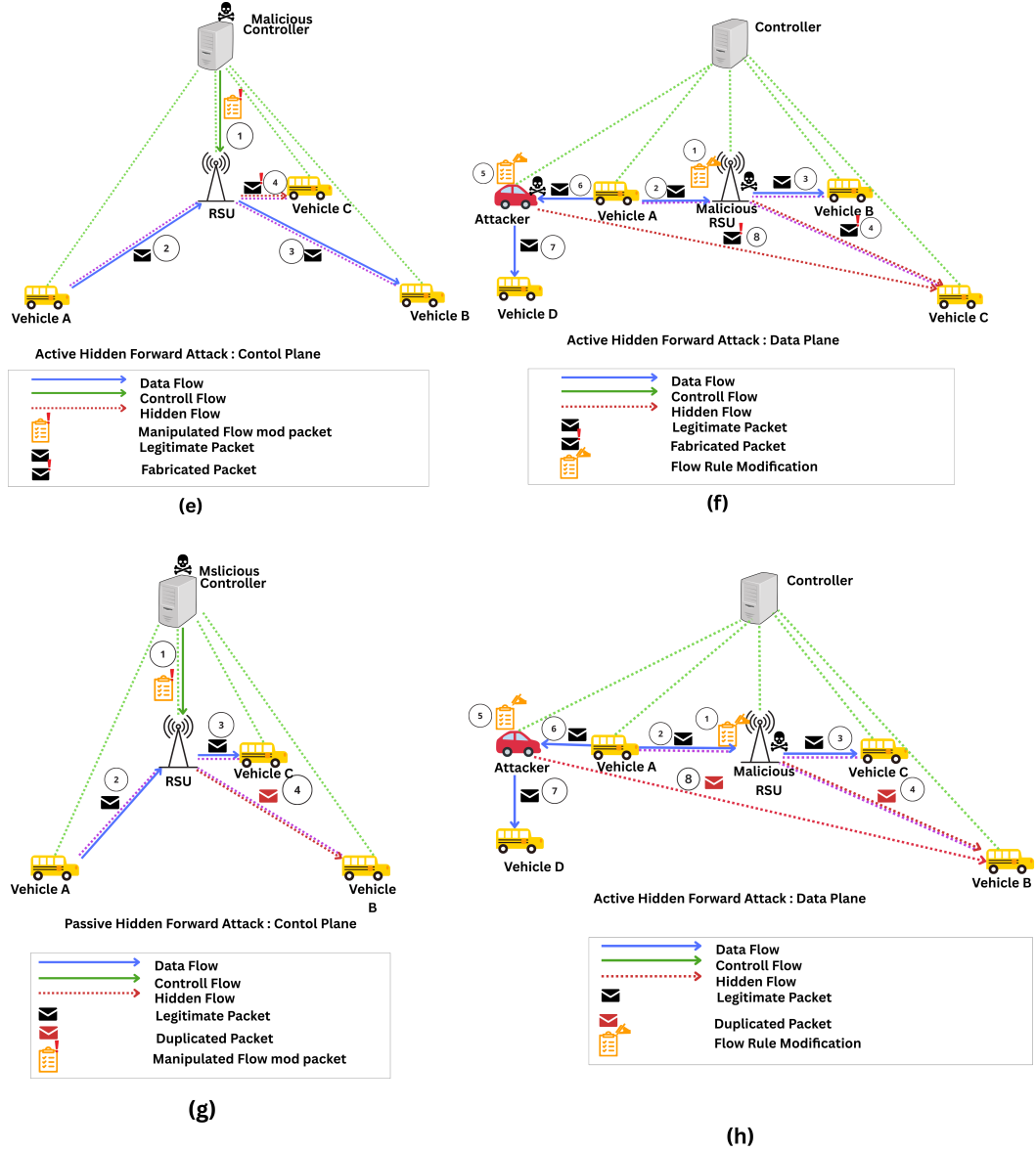


Figure 3.3: Hidden Forwarding Attack Versions

...

3.3.3 Mobility-Induced Threat Amplification in SDVN

The eight attack variants defined in Sections ?? and ?? are not merely transposed from static SDN to a vehicular setting; vehicular mobility structurally amplifies each attack class through four mechanisms that must be explicitly modeled to derive accurate detection thresholds and cryptographic verification scopes.

Mechanism 1: Handoff masking of delay signatures. At highway speeds of 80–120 km/h, a vehicle traverses a typical RSU coverage zone of 300–500 m in approximately 9–22 s, inducing legitimate handoff latencies of 50–300 ms during flow-rule recomputation and reinstallation [?]. An adversary executing Attack Variants 1 and 2 (Selective Time Delay, control-plane and data-plane) can inject delays within this range, rendering attack-induced delay statistically indistinguishable from legitimate handoff jitter. Let δ_h denote the handoff latency distribution

and δ_a the attacker-injected delay; detection is feasible only when the two distributions are separable, i.e., when their Bhattacharyya distance

$$D_B(\delta_h, \delta_a) = -\ln \left(\int \sqrt{p_h(x) p_a(x)} dx \right) > \epsilon \quad (3.1)$$

for some detection threshold ϵ . Existing static-SDN detectors implicitly assume $\delta_h = 0$, making them blind to this overlap.

Mechanism 2: Topology change exploiting control-plane staleness.

The SDVN controller maintains its network view through periodic LLDP topology discovery messages, typically every 1–5 s [?]. Between discovery cycles, the controller’s topology view is stale by definition. In static SDN this staleness is negligible; in SDVN, vehicles enter and leave RSU coverage zones continuously, meaning the controller’s forwarding graph can diverge substantially from the true network state within a single discovery interval. Attackers exploit this window in two ways. First, for selective time delay attacks, a compromised RSU can exploit stale routing decisions that direct traffic through a suboptimal multi-hop path, adding propagation delay that is indistinguishable from attack-induced delay since the controller itself expects elevated latency due to its outdated topology view. Second, for hidden forwarding attacks, a malicious node can install covert forwarding rules during a topology update cycle; these rules are not present in the controller’s stale view and therefore escape flow-rule consistency verification. The effective detection window narrows with the topology update interval T_{topo} ; the probability that a malicious rule survives undetected for k consecutive discovery cycles is

$$P_{evade}(k) = (1 - P_{detect})^k \quad (3.2)$$

where P_{detect} is the per-cycle detection probability. In high-mobility SDVN, frequent topology changes reduce P_{detect} per cycle by invalidating the reference forwarding model used for comparison, making $P_{evade}(k)$ unacceptably large even for moderate k .

Mechanism 3: Flow-rule churn camouflage of TCAM exhaustion.

In SDVN, legitimate mobility-driven flow installations at busy intersections reach rates of hundreds of **FlowMod** messages per minute as vehicles continuously enter and leave RSU coverage zones. The slow TCAM exhaustion variants (Attacks 3 and 4) operate at rates of 3.2–40 packets/s [?], which fall within the lower tail of the legitimate installation rate distribution. Let $\lambda_l(t)$ denote the legitimate flow installation rate as a function of vehicle density $\rho(t)$, and λ_a the attack installation rate. Detection requires identifying λ_a as anomalous within the observed total rate $\lambda_{obs}(t) = \lambda_l(t) + \lambda_a$. A static threshold λ_{thresh} calibrated for static SDN will generate unacceptably high false positive rates in SDVN because $\lambda_l(t)$ varies significantly with road density. This requires vehicle-density-normalized TCAM installation rate monitoring

$$\hat{\lambda}_a(t) = \lambda_{obs}(t) - \mathbb{E}[\lambda_l(t) | \rho(t)] \quad (3.3)$$

as part of the rule-based detection signature for Variants 3 and 4, where $\mathbb{E}[\lambda_l(t) | \rho(t)]$ is estimated from the predicted vehicle density at the RSU.

Mechanism 4: Short observation windows for hidden forwarding detection. Flow conservation-based hidden forwarding detectors such as FADE and

HSA require sufficient packet samples over a flow’s lifetime to detect duplication or path deviation. In SDVN, a vehicle resides within a single RSU’s observation window for only 9–22s at highway speed, and each topology change effectively resets the observation window for multi-hop path validation since the reference header space must be recomputed. Let N_{obs} denote the number of packets observed within a single RSU zone crossing and N_{min} the minimum sample size for reliable flow conservation testing at significance level α . The condition

$$N_{obs} = r \cdot t_{zone} < N_{min}(\alpha) \quad (3.4)$$

where r is the per-flow packet rate and t_{zone} is the zone residence time, is frequently violated under highway mobility, rendering flow-conservation approaches statistically unreliable. This fundamentally motivates the use of cryptographic per-packet ZKP-based behavioral verification as the primary hidden forwarding mitigation mechanism, since ZKP verification is per-packet and requires no observation window, complemented by federated LSTM sequence modeling that can detect anomalous patterns over short windows without requiring flow-level aggregation.

3.4 High-Level Functional Block Diagram

The defense mechanism handles communication through a number of functional stages to guarantee network trust and real-time detection as shown in Figure 3.4

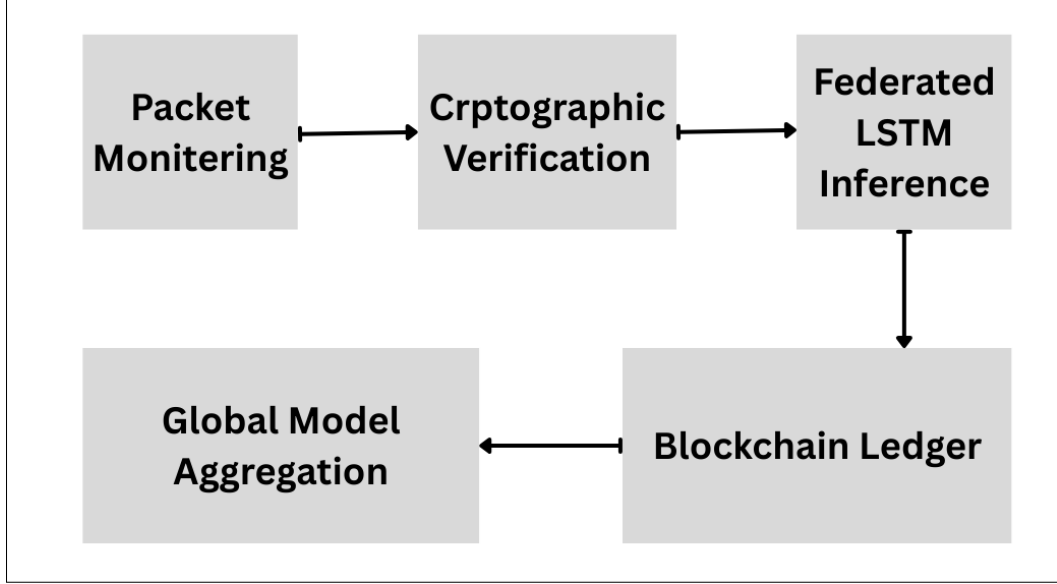


Figure 3.4: Block Diagram

1. Layer 1: Packet Monitoring: This involves capturing sequential packet data, including timing and routing histories [9]
2. Layer 2: Crptographic Verification: Uses digital signatures and secure hashing to ensure the integrity and authenticity of every packet [20]
3. Layer 3: To find odd patterns or anomalies, local long short-term memory (LSTM) models examine time-dependent data. [9]
4. Layer 4:Blockchain Ledger: To remove single points of failure, securely record network events and reach agreement on a tamper-proof distributed ledger. [22] [23]
5. Layer 5: To improve the global detection model while maintaining privacy, local model updates are combined via a blockchain-protected server.

3.4.1 Role of Cryptography: Pre-Detection or Post-Detection?

A critical architectural question is whether the proposed cryptographic layer—EdDSA per-packet signing, aggregate signature verification, and ZKP-based behavioral proofs—independently mitigates both attack classes or whether detection remains necessary. We analyze each attack class separately.

For hidden forwarding attacks (Variants 5–8), the cryptographic layer provides strong mitigation: a forwarding node cannot produce a valid ZKP demonstrating legitimate next-hop forwarding unless it actually forwarded the packet to an authorized destination and obtained a valid receipt signature from that destination.

Table 3.1: Detection and Mitigation Logic Separation Across Dual Modes

Component	Lightweight Mode	Full Mode
Detection	Rule-based attack signature matching with mobility-adjusted thresholds (OBU-deployable)	Federated LSTM anomaly detection at RSU edge nodes
Mitigation	HMAC-based lightweight packet authentication; local flow-rule quarantine	EdDSA + aggregate signatures + ZKP behavioral proofs; blockchain-enforced trust penalties
Blockchain	Lightweight event logging only (detection support)	Smart contract-enforced trust scoring, node isolation, and model validation (detection + mitigation)

Since the controller and blockchain verify this proof before updating trust, hidden forwarding is *cryptographically prevented* for nodes participating honestly in the signing protocol. However, a Sybil attacker generating colluding fake next-hop identities could potentially produce false receipt signatures; detection therefore remains necessary as a second line of defense.

For selective time delay attacks (Variants 1–4), the cryptographic layer does *not* independently prevent the attack. An attacker can delay a packet by the desired interval δ_a and then sign it with the correct timestamp *after* the delay, producing a valid signature that reflects the manipulated timestamp. The ZKP demonstrating forwarding within delay threshold Δ_{max} would fail in this case, but only if the receiving next-hop records an independent timestamp at reception—which is the mechanism that enables detection. Therefore, for timing attacks, cryptography acts as a *detection enabler* applied *before* the detection logic: the EdDSA timestamp binding and ZKP generation provide the tamper-evident evidence on which the rule-based and LSTM detectors operate.

Consequently, the architecture applies cryptography *before* detection for timing attacks (as evidence generation) and *as primary mitigation* for forwarding attacks (with detection as fallback). Blockchain serves *both* roles: it immutably logs detection events (detection support) and enforces trust penalties and node isolation via smart contracts (mitigation).

...

3.4.2 Framework Architecture: Detection and Mitigation Separation

The proposed framework cleanly separates detection logic from mitigation logic across two operational modes as summarized in Table 3.1.

The lightweight mode is designed for resource-constrained OBUs operating under computational budgets of <100 MHz and <256 MB RAM; the full mode targets RSU edge nodes with GPU-capable edge computing. Both modes are simultaneously active in the network; OBUs run lightweight detection and escalate suspicious events to the nearest RSU, which applies full-mode analysis. This escalation mechanism ensures that no attack variant evades detection due to resource constraints at the detecting node.

...

3.4.3 Attack Signature Identification

The rule-based lightweight detection engine operates on distinguishing signatures—observable, measurable behavioral patterns that characterize each attack variant and differentiate it from benign activity. We derive one primary signature per variant, accounting for mobility-induced masking identified in Section 3.3.3.

Signature S1 (Selective Time Delay, Control Plane): Safety-critical packets (identified by DSCP priority marking or BSM message type) originating from the same source vehicle v experience per-hop forwarding delays at RSU r that consistently exceed the mobility-adjusted baseline $\bar{\delta}_r(t)$ by more than k standard deviations, while best-effort traffic from the same RSU remains within baseline. The selective nature (only high-priority packets are delayed) is the distinguishing feature separating Attack 1 from generic congestion.

$$\text{S1: } \delta_p(v, r, t) > \bar{\delta}_r(t) + k\sigma_r(t) \quad \wedge \quad \text{Priority}(p) = \text{HIGH} \quad (3.5)$$

Signature S2 (Selective Time Delay, Data Plane): The inter-node forwarding delay at an intermediate vehicle node u exceeds the expected single-hop propagation and processing delay $\delta_{prop} + \delta_{proc}$ by more than Δ_{max} , as evidenced by a ZKP failure at the next hop (next hop's independently recorded reception timestamp contradicts the forwarding node's claimed timestamp).

$$\text{S2: } t_{u+1}^{recv} - t_u^{fwd} > \Delta_{max} \quad \wedge \quad \pi_{delay}(u) = \perp \quad (3.6)$$

Signature S3 (Slow TCAM Exhaustion, Control Plane): The FlowMod installation rate $\lambda_{FM}(r, t)$ at RSU r from the controller exceeds the density-normalized expected rate by threshold λ_{thresh} , while the installed rules do not correspond to any active vehicle flow in the network topology.

$$\text{S3: } \lambda_{FM}(r, t) - \mathbb{E}[\lambda_t(t) \mid \rho(t)] > \lambda_{thresh} \quad \wedge \quad \nexists v : \text{flow}(r) \in \mathcal{F}_{active}(v) \quad (3.7)$$

Signature S4 (Slow TCAM Exhaustion, Data Plane): A single source vehicle v_{atk} generates a stream of packets with unique 5-tuples at rate λ_{atk} causing repeated PACKET_IN floods to the controller, with TCAM utilization $U_{TCAM}(r, t)$ exceeding threshold U_{thresh} while v_{atk} has no legitimate active session requiring such flow diversity.

$$\text{S4: } \lambda_{PI}(v_{atk}, r, t) > \lambda_{PI,thresh} \quad \wedge \quad U_{TCAM}(r, t) > U_{thresh} \quad (3.8)$$

Signature S5 (Active Hidden Forwarding, Control Plane): A FlowMod message from the controller installs a forwarding rule at RSU r with an output

port mapping to a destination d' not present in the authorized forwarding policy $\mathcal{P}$ for the corresponding source-destination pair (s, d) .

$$\text{S5: } d' \notin \mathcal{P}(s, d) \quad \wedge \quad \text{FlowMod}(r) \text{ installed } (s \rightarrow d') \quad (3.9)$$

Signature S6 (Active Hidden Forwarding, Data Plane): A packet with message identifier `msg_id` is observed arriving at two distinct destination nodes d and d' within observation window W , where only one is the authorized destination per the current flow table.

$$\text{S6: } \exists d, d' : d \neq d' \wedge \text{msg_id} \in \mathcal{R}(d, W) \wedge \text{msg_id} \in \mathcal{R}(d', W) \quad (3.10)$$

Signature S7 (Passive Hidden Forwarding, Control Plane): Traffic volume at an unauthorized listening node d' increases monotonically over window W without any corresponding `FlowMod` from the controller authorizing flows to d' , indicating a covert duplication channel established by a malicious controller application.

$$\text{S7: } \frac{d}{dt} \text{Vol}(d', t) > \epsilon_{vol} \quad \wedge \quad \nexists \text{FlowMod}(r) : \text{dst} = d' \quad (3.11)$$

Signature S8 (Passive Hidden Forwarding, Data Plane): The aggregate signature verification `VERIFYAGG` succeeds for the legitimate path $s \rightarrow \dots \rightarrow d$, but the ZKP $\pi_{hop}(u)$ produced by intermediate node u fails next-hop legitimacy verification, indicating u forwarded to an additional unauthorized destination while correctly serving the legitimate one.

$$\text{S8: } \text{VERIFYAGG}(\sigma_{agg}, \mathbf{pk}, \mathbf{m}) = 1 \quad \wedge \quad \pi_{hop}(u) = \perp \quad (3.12)$$

...

3.4.4 Lightweight Rule-Based Detection Engine

The lightweight detection engine evaluates signatures S1–S8 at each node using mobility-adjusted parameters computed locally without requiring controller coordination. For timing attacks, the mobility-adjusted baseline delay at RSU r at time t is estimated as a function of current vehicle density $\rho(t)$ and mean vehicle speed $\bar{v}(t)$:

$$\bar{\delta}_r(t) = \delta_0 + \alpha_\rho \cdot \rho(t) + \alpha_v \cdot \bar{v}(t)^{-1} \quad (3.13)$$

where δ_0 is the static propagation baseline and α_ρ , α_v are density and speed sensitivity coefficients fitted from offline mobility traces. The standard deviation $\sigma_r(t)$ is estimated using an exponentially weighted moving average:

$$\sigma_r^2(t) = \beta \cdot \sigma_r^2(t-1) + (1-\beta) \cdot (\delta_r(t) - \bar{\delta}_r(t))^2 \quad (3.14)$$

A packet p is flagged by the S1 rule if:

$$f_{S1}(p, r, t) = \begin{cases} 1 & \delta_p > \bar{\delta}_r(t) + k\sigma_r(t) \wedge \text{Priority}(p) = \text{HIGH} \\ 0 & \text{otherwise} \end{cases} \quad (3.15)$$

For TCAM exhaustion detection, the anomalous installation rate estimator from (3.3) triggers a flag when:

$$f_{S3,S4}(r, t) = \begin{cases} 1 & \hat{\lambda}_a(r, t) > \lambda_{thresh} \wedge U_{TCAM}(r, t) > U_{thresh} \\ 0 & \text{otherwise} \end{cases} \quad (3.16)$$

The composite lightweight detection decision for node v at time t is:

$$D_{light}(v, t) = \bigvee_{i=1}^8 f_{Si}(v, t) \quad (3.17)$$

where f_{Si} is the rule function for signature i . A positive detection triggers escalation to the nearest RSU for full-mode analysis.

3.4.5 Hybrid Cryptographic Integrity Layer

Per-Packet EdDSA Signing

Each forwarding node v_i with private key sk_i signs every forwarded packet by computing:

$$\sigma_i = \text{EDDSA.SIGN}(sk_i, H(\text{msg_id} \parallel ts_i \parallel \eta_i \parallel \text{next_hop}_i \parallel z_i)) \quad (3.18)$$

where ts_i is v_i 's local timestamp at forwarding, η_i is a cryptographic nonce preventing replay, and $z_i \in \mathcal{Z}_i$ is the current RSU zone identifier that binds the signature to the vehicle's geographic context, preventing cross-zone replay attacks. The signed message binds identity, time, routing decision, and mobility context simultaneously.

Aggregate Signature Construction

As a packet traverses n hops $v_1, v_2, \dots, v_n$, individual signatures are aggregated using a BLS-compatible scheme:

$$\sigma_{agg} = \prod_{i=1}^n \sigma_i \quad (3.19)$$

The RSU or controller verifies the aggregate in constant time:

$$\text{VERIFYAGG}(\sigma_{agg}, \{pk_i\}_{i=1}^n, \{m_i\}_{i=1}^n) = \hat{e}(\sigma_{agg}, g_2) \stackrel{?}{=} \prod_{i=1}^n \hat{e}(H(m_i), pk_i) \quad (3.20)$$

where $\hat{e}$ is the bilinear pairing and g_2 is the generator of $\mathbb{G}_2$. This reduces per-hop verification overhead from $O(n)$ to $O(1)$ at the RSU, which is critical for high-throughput vehicular networks.

ZKP for Timing Compliance

Each forwarding node v_i generates a zero-knowledge proof $\pi_{delay,i}$ demonstrating that the forwarding delay was within threshold Δ_{max} without revealing the raw timestamps:

$$\pi_{delay,i} = \text{ZKP.PROVE}\left(\{t_i^{recv}, t_i^{fwd}, r_i\}; C_{delay} : t_i^{fwd} - t_i^{recv} \leq \Delta_{max} \wedge H(r_i) = c_i\right) \quad (3.21)$$

where r_i is a blinding factor, $c_i = H(r_i)$ is its public commitment, and C_{delay} is the arithmetic circuit encoding the delay constraint. The proof $\pi_{delay,i}$ is publicly verifiable without exposing t_i^{recv} or t_i^{fwd} .

ZKP for Next-Hop Legitimacy

Each forwarding node v_i additionally proves that its chosen next hop v_{i+1} is authorized per the current routing policy and produced a valid receipt signature σ_{i+1}^{rcpt} .

$$\pi_{hop,i} = \text{ZKP.PROVE}\left(\{\sigma_{i+1}^{rcpt}, \mathcal{P}_i\}; C_{hop} : \text{VERIFY}(\sigma_{i+1}^{rcpt}, pk_{i+1}) = 1 \wedge v_{i+1} \in \mathcal{P}_i\right) \quad (3.22)$$

where $\mathcal{P}_i$ is v_i 's local copy of the authorized next-hop set for the current flow, committed to the blockchain at flow setup. The combined proof $\pi_i = (\pi_{delay,i}, \pi_{hop,i})$ is forwarded alongside σ_i to the controller/blockchain for verification.

Lightweight Cryptography (HMAC)

For resource-constrained OBUs operating in lightweight mode, EdDSA is replaced with an HMAC using a pre-shared symmetric key k_i established during RSU association:

$$\tau_i = \text{HMAC}(k_i, \text{msg_id} \parallel ts_i \parallel \eta_i) \quad (3.23)$$

While HMAC provides integrity and authenticity, it does not support aggregate verification or ZKP generation; lightweight nodes therefore contribute only to rule-based detection and rely on RSU-level full verification for mitigation.

Communication Overhead Comparison

The total cryptographic overhead per packet in each mode is:

$$O_{full} = |\sigma_i| + |\sigma_{agg}| + |\pi_{delay}| + |\pi_{hop}| \quad (3.24)$$

$$O_{light} = |\tau_i| \quad (3.25)$$

With EdDSA signatures of 64 bytes, BLS aggregate signatures of 48 bytes, and Groth16 ZKP proofs of ≈ 192 bytes, $O_{full} \approx 368$ bytes per packet; with HMAC-SHA256, $O_{light} = 32$ bytes. The $11.5\times$ overhead reduction in lightweight mode makes it viable for OBUs on constrained radio channels.

3.4.6 Federated LSTM Anomaly Detection

Local LSTM Model

Each RSU r_k trains a local LSTM model on its observed packet sequences. The input feature vector at timestep t for flow f is:

$$\mathbf{x}_t^{(f)} = [\delta_t, \lambda_{PI,t}, U_{TCAM,t}, \mathbb{K}[\pi_{delay} = \perp], \mathbb{K}[\pi_{hop} = \perp], \rho_t, \bar{v}_t]^T \quad (3.26)$$

where the ZKP failure indicators $\mathbb{K}[\pi = \perp]$ are novel mobility-cryptographic features that encode cryptographic evidence of attacks directly into the LSTM input. The hidden state evolves as:

$$\mathbf{h}_t = \text{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1}; \mathbf{W}_{local}^{(k)}) \quad (3.27)$$

The anomaly score for timestep t is the reconstruction error of an autoencoder-mode LSTM:

$$\mathcal{A}_t^{(k)} = \|\mathbf{x}_t - \hat{\mathbf{x}}_t\|_2^2 \quad (3.28)$$

A detection event is triggered when:

$$D_{LSTM}^{(k)}(t) = \begin{cases} 1 & \mathcal{A}_t^{(k)} > \theta^{(k)} \\ 0 & \text{otherwise} \end{cases} \quad (3.29)$$

where the RSU-specific threshold $\theta^{(k)}$ is computed from the local training distribution:

$$\theta^{(k)} = \mu_{\mathcal{A}}^{(k)} + z_{\alpha} \cdot \sigma_{\mathcal{A}}^{(k)} \quad (3.30)$$

and z_{α} is the z -score corresponding to false positive rate α .

Federated Aggregation with Byzantine Robustness

Since vehicular nodes may be compromised, standard FedAvg is replaced with a Byzantine-robust aggregation strategy. The global model weights are updated as:

$$\mathbf{W}_{global}^{(t+1)} = \frac{\sum_{k=1}^K n_k \cdot \mathbf{W}_{local}^{(k)}(t) \cdot \mathbb{K}\left[d\left(\mathbf{W}_{local}^{(k)}, \tilde{\mathbf{W}}\right) < \gamma\right]}{\sum_{k=1}^K n_k \cdot \mathbb{K}\left[d\left(\mathbf{W}_{local}^{(k)}, \tilde{\mathbf{W}}\right) < \gamma\right]} \quad (3.31)$$

where $\tilde{\mathbf{W}}$ is the coordinate-wise median of submitted weights, $d(\cdot, \cdot)$ is Euclidean distance, and γ is an outlier rejection threshold. This Krum-inspired filtering excludes model updates from RSUs whose weights deviate significantly from the consensus—rejecting gradient poisoning by compromised RSU nodes. The smart contract on the blockchain verifies the hash of submitted weights before inclusion in aggregation:

$$\mathbb{K}_{BC}^{(k)} = \text{SC.VERIFYMODELHASH}\left(H(\mathbf{W}_{local}^{(k)}), \text{COMMITTEDHASH}^{(k)}\right) \quad (3.32)$$

3.4.7 Blockchain Architecture: Design Beyond Storage

The blockchain component is not applied merely as a tamper-proof ledger; it is architecturally designed with three specific SDVN requirements in mind.

Two-tier chain structure. A local RSU-level permissioned chain (RSU-chain) operates with low-latency Practical Byzantine Fault Tolerant (PBFT) consensus among neighboring RSUs, tolerating $f < n/3$ Byzantine RSU nodes. The global chain anchors RSU-chain Merkle roots periodically:

$$H_{anchor}^{(r)} = H(H_{root}^{RSU} \parallel ts_{anchor} \parallel H_{prev}^{global}) \quad (3.33)$$

This design limits global consensus overhead to anchor events (≈ 1 per minute) while enabling sub-second local logging latency for real-time detection support.

Smart contract for automated trust management. Each node v maintains a blockchain-recorded trust score $T_v \in [0, 1]$ updated by a smart contract upon each ZKP and aggregate signature verification outcome:

$$T_v^{(t+1)} = \begin{cases} \min(1, T_v^{(t)} + \Delta_r) & \text{if } \text{VERIFYAGG} = 1 \wedge \pi_i = \top \\ \max(0, T_v^{(t)} - \Delta_p) & \text{if } \text{VERIFYAGG} = 0 \vee \pi_i = \perp \end{cases} \quad (3.34)$$

where Δ_r and Δ_p are the reward and penalty increments. When $T_v^{(t)} < T_{min}$, the smart contract automatically emits a QUARANTINE event, instructing all RSUs to drop flows originating from v :

$$\text{SC.QUARANTINE}(v) \Leftarrow T_v^{(t)} < T_{min} \quad (3.35)$$

Flow-rule commitment scheme. At flow setup, the controller commits the authorized forwarding policy $\mathcal{P}(s, d)$ to the blockchain:

$$C_{\mathcal{P}} = \text{COMMIT}(H(\mathcal{P}(s, d)) \parallel ts_{setup}) \quad (3.36)$$

ZKP next-hop proofs reference this commitment, preventing the controller from secretly modifying $\mathcal{P}(s, d)$ after flow installation without detection. This is the key design distinction from prior blockchain-for-SDN works (FTISCON, BlockREV) which verify rules only at deployment time; here the commitment persists and is referenced by every forwarded packet throughout the flow's lifetime.

....

...

...Figure 1

Figure 2

figure 3

Figure 4

3.5 Data Collection

The attacks will be simulated in ns-3 and data will be collected using simulated environment.

Algorithm 1 Lightweight Rule-Based Attack Detection (LRAD)

```

1: procedure LRAD( $p, v, r, t, \rho, \bar{v}$ )
2:    $\bar{\delta} \leftarrow \delta_0 + \alpha_\rho \cdot \rho + \alpha_v \cdot \bar{v}^{-1}$  ▷ Eq. (3.13)
3:    $\sigma_r \leftarrow \text{EWMA}(\delta_r, \beta)$  ▷ Eq. (3.14)
4:    $\hat{\lambda}_a \leftarrow \lambda_{obs} - \mathbb{E}[\lambda_l | \rho]$  ▷ Eq. (3.3)
5:   for  $i \leftarrow 1$  to 8 do
6:      $flag_i \leftarrow \text{EVAL\_SIG}(S_i, p, v, r, t, \bar{\delta}, \sigma_r, \hat{\lambda}_a)$ 
7:   end for
8:    $D_{light} \leftarrow \bigvee_{i=1}^8 flag_i$  ▷ Eq. (3.17)
9:   if  $D_{light} = 1$  then
10:     $\text{ESCALATE}(p, v, r, \{flag_i\})$  ▷ Send to RSU for full-mode analysis
11:  end if
12:  return  $D_{light}$ 
13: end procedure

```

Algorithm 2 Full Cryptographic Integrity Pipeline (FCIP)

```

1: procedure FCIP( $p, v_i, sk_i, z_i, \Delta_{max}, \mathcal{P}_i$ )
2:    $\eta_i \leftarrow \text{RAND}()$  ▷ Fresh nonce
3:    $\sigma_i \leftarrow \text{EdDSA.Sign}(sk_i, H(\text{msg\_id} \| ts_i \| \eta_i \| nh_i \| z_i))$  ▷ Eq. (3.18)
4:    $\sigma_{agg} \leftarrow \text{AGG.Combine}(\sigma_{agg}^{prev}, \sigma_i)$  ▷ Eq. (3.19)
5:    $\pi_{delay} \leftarrow \text{ZKP.Prove}(C_{delay}, t_i^{recv}, t_i^{fwd}, r_i)$  ▷ Eq. (3.21)
6:    $\sigma^{rcpt} \leftarrow \text{AWAIT\_RECEIPT}(v_{i+1})$ 
7:    $\pi_{hop} \leftarrow \text{ZKP.Prove}(C_{hop}, \sigma^{rcpt}, \mathcal{P}_i)$  ▷ Eq. (3.22)
8:    $\text{FORWARD}(p, \sigma_{agg}, \pi_{delay}, \pi_{hop})$ 
9:   return  $(\sigma_{agg}, \pi_{delay}, \pi_{hop})$ 
10: end procedure

```

Algorithm 3 Byzantine-Robust Federated Aggregation (BRFA)

```

1: procedure BRFA( $\{\mathbf{W}^{(k)}\}_{k=1}^K, \{n_k\}, \gamma$ )
2:    $\tilde{\mathbf{W}} \leftarrow \text{COORD\_MEDIAN}(\{\mathbf{W}^{(k)}\})$  ▷ Coordinate-wise median
3:   for  $k \leftarrow 1$  to  $K$  do
4:      $\mathbb{K}_{BC}^{(k)} \leftarrow \text{SC.VerifyModelHash}(\mathbf{W}^{(k)})$  ▷ Eq. (3.32)
5:      $\omega_k \leftarrow n_k \cdot \mathbb{K}_{BC}^{(k)} \cdot \mathbb{K}[d(\mathbf{W}^{(k)}, \tilde{\mathbf{W}}) < \gamma]$ 
6:   end for
7:    $\mathbf{W}_{global} \leftarrow \frac{\sum_k \omega_k \mathbf{W}^{(k)}}{\sum_k \omega_k}$  ▷ Eq. (3.31)
8:    $\text{BC.COMMIT}(H(\mathbf{W}_{global}))$ 
9:   return  $\mathbf{W}_{global}$ 
10: end procedure

```

Algorithm 4 Blockchain Trust Management and Mitigation (BTMM)

```

1: procedure BTMM( $v, \sigma_{agg}, \pi_{delay}, \pi_{hop}, \mathbf{pk}, \mathbf{m}$ )
2:    $b_{agg} \leftarrow \text{VerifyAgg}(\sigma_{agg}, \mathbf{pk}, \mathbf{m})$  ▷ Eq. (3.20)
3:    $b_{\pi} \leftarrow \text{ZKP.Verify}(\pi_{delay}) \wedge \text{ZKP.Verify}(\pi_{hop})$ 
4:   if  $b_{agg} \wedge b_{\pi}$  then
5:      $T_v \leftarrow \min(1, T_v + \Delta_r)$  ▷ Eq. (3.34)
6:   else
7:      $T_v \leftarrow \max(0, T_v - \Delta_p)$  ▷ Eq. (3.34)
8:      $\text{BC.LOG}(v, \neg b_{agg}, \neg b_{\pi}, ts_{event})$ 
9:   end if
10:  if  $T_v < T_{min}$  then
11:     $\text{SC.Quarantine}(v)$  ▷ Eq. (3.35)
12:  end if
13:  return  $T_v$ 
14: end procedure

```

Chapter 4

Timeline and Resource Required

...

4.0.1 Baseline Selection and Benchmarking Rationale

External Baselines

Three post-2020 baselines are selected that directly address selective time delay or hidden forwarding attacks in SDN or SDVN.

B1 — FSDM [?]: Closest prior art for control-plane saturation-induced timing delay (Variants 1–3). Represents the best centralized entropy-based approach; comparison exposes its failure under vehicular mobility and compromised controller conditions.

B2 — SFTO-Guard [?]: Most recent dedicated defense against slow-rate TCAM exhaustion (Variants 3–4). Comparison isolates the performance degradation of static ML-based thresholds under vehicular topology dynamics.

B3 — FADE [?]: State-of-the-art flow-conservation-based forwarding anomaly detector covering packet duplication and path deviation (Variants 5–8). Comparison directly demonstrates the short-observation-window limitation (Eq. (3.4)) of flow-conservation approaches under vehicular mobility.

Internal Ablation Baselines

Component-ablated variants isolate each contribution’s individual impact:

- **A1 — Rule-only**: Lightweight rule engine (Alg. 1) alone; no LSTM, no cryptographic layer.
- **A2 — LSTM-only**: Federated LSTM (Alg. 3) alone; no rule pre-filtering, no ZKP features in input vector (3.26).
- **A3 — No-ZKP**: Full system with EdDSA and aggregate signatures but without ZKP proofs; isolates ZKP contribution to both detection and mitigation.
- **A4 — No-BC**: Full system without blockchain trust management; detection retained but automated quarantine (Eq. (3.35)) and model validation (Eq. (3.32)) disabled.
- **A5 — Full MOBIGUARD**: Complete proposed framework.

Primary Performance Metrics

Four metrics are selected to evaluate both detection quality and mitigation effectiveness.

M1 — Matthews Correlation Coefficient (MCC): Primary detection quality metric, selected over accuracy and F1 because it remains reliable under the class imbalance inherent in attack detection datasets where benign packets vastly outnumber attack packets:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (4.1)$$

M2 — Per-Variant Detection Rate (DR): True positive rate evaluated independently for each of the eight attack variants, ensuring no variant is masked by aggregate accuracy figures.

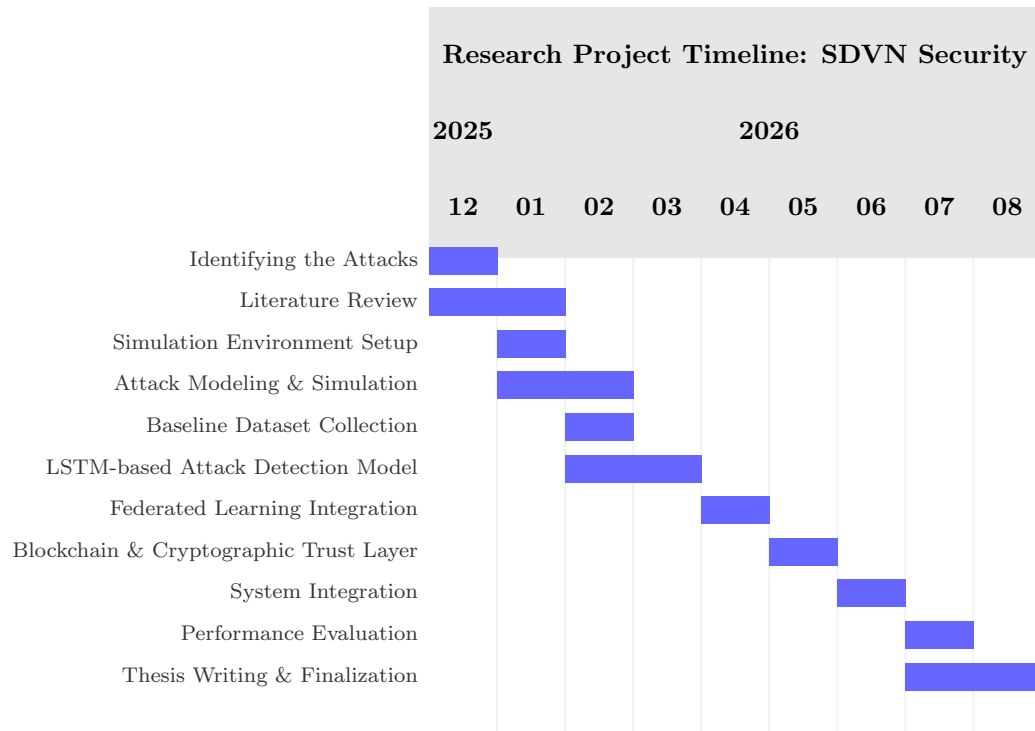
M3 — False Positive Rate (FPR): Critical in vehicular safety contexts where false alarms trigger mitigation actions that themselves degrade network performance; evaluated separately for lightweight and full modes.

M4 — Mitigation Latency (L_{mit}): Time from attack onset to smart contract quarantine execution (Eq. (3.35)), measured against the 100 ms safety-critical latency bound; directly evaluates mitigation effectiveness rather than detection alone.

M5 — Packet Delivery Ratio (PDR): Fraction of safety-critical packets successfully delivered to the final destination across the full multihop path, evaluated under active attack with and without MOBIGUARD. PDR directly captures the combined impact of both selective time delay (packets dropped after timeout) and hidden forwarding (packets corrupted or rerouted) on end-to-end delivery across all intermediate hops.

M6 — End-to-End Latency (L_{e2e}): Mean per-packet latency accumulated across all hops from source to final destination, compared against the 100 ms safety-critical bound. Evaluated under attack and post-mitigation to quantify the latency restoration effectiveness of the proposed framework relative to the three external baselines.

4.1 Timeline



4.2 Resource Required

To execute the high-fidelity simulations and train the Federated LSTM models, high-performance computing resources are required.

Since this research will be done using leexisting university computing infrastructure, there is no direct financial cost.

Chapter 5

Conclusion

Software Defined Vehicular Networks (SDVNs) play a significant role in modern intelligent transport systems. They offer flexible, efficient, and adaptive communication among vehicles. Although the programmable and centrally controlled aspects of SDVNs make them very potent, they also introduce a significant security issue. Among all the other attacks, Selective Time Delay Attack and Hidden Forwarding Attack are considered extremely harmful. They target communication that is critical for safety without properly resulting in network malfunctions. Hence, traditional security solutions fail to identify such attacks.

The proposed research study addresses all these problems with a comprehensive security framework that consists of Blockchain, cryptography techniques, and Federated LSTM Anomaly Detection. Contrary to all other proposed systems, which always work on a Centralized Architecture, the new framework will operate with a Zero-Trust Approach. This would make it possible for the system to identify as well as authenticate malicious behavior from any element in the network, including controllers, RSUs, or vehicles, without relying on a central point of trust.

The proposed approach aims at providing a secured environment by protecting privacy while at the same time providing a low-latency service for vehicular safety applications. The proposed approach will be demonstrated on a realistic simulation environment for SDVN in order to test the performance on a variety of attack scenarios.

Finally, it can be concluded that this paper attempts at offering a privacy-preserving security platform that is also dependable for next-generation Software Defined Vehicular Networks. With the ability to counter Selective Time Delay Attacks as well as Hidden Forwarding attacks within the same framework, it will enable the development of safer vehicles on the road.

References

- [1] N. Cardona, E. Coronado, S. Latre, R. Riggio, and J. M. Marquez-Barja, “Software-defined vehicular networking: Opportunities and challenges,” *IEEE Access*, vol. 8, pp. 219971–219995, 2020.
- [2] P. A. D. S. N. Wijesekara, K. L. K. Sudheera, G. G. N. Sandamali, and P. H. J. Chong, “An optimization framework for data collection in software defined vehicular networks,” *Sensors*, vol. 23, no. 3, 2023.
- [3] K. S. Kalupahana Liyanage, M. Ma, and P. H. Joo Chong, “Controller placement optimization in hierarchical distributed software defined vehicular networks,” *Computer Networks*, vol. 135, pp. 226–239, 2018.
- [4] T. A. Pascoal, I. E. Fonseca, and V. Nigam, “Slow denial-of-service attacks on software defined networks,” *Computer Networks*, vol. 173, p. 107223, 2020.
- [5] O. S. Al-Heety, Z. Zakaria, M. Ismail, M. M. Shakir, S. Alani, and H. Alsari-
era, “A comprehensive survey: Benefits, services, recent works, challenges, security, and use cases for sdn-vanet,” *IEEE Access*, vol. 8, pp. 91028–91047, 2020.
- [6] F. Bannour, S. Souihi, and A. Mellouk, “Distributed sdn control: Survey, taxonomy, and challenges,” *IEEE Communications Surveys & Tutorials*, vol. 20, no. 1, pp. 333–354, 2018.
- [7] I. Yaqoob, I. Ahmad, E. Ahmed, A. Gani, M. Imran, and N. Guizani, “Overcoming the key challenges to establishing vehicular communication: Is sdn the answer?,” *IEEE Communications Magazine*, vol. 55, no. 7, pp. 128–134, 2017.
- [8] I. Sumra, J.-L. Ab Manan, and H. Hasbullah, “Timing attack in vehicular network,” 07 2011.
- [9] A. Qammar, A. Karim, H. Ning, and J. Ding, “Securing federated learning with blockchain: a systematic literature review,” *Artif. Intell. Rev.*, vol. 56, no. 5, pp. 3951–3985, 2023.
- [10] A. A. Mohammadi, P. Kazemian, and M. R. Pakravan, “Detecting malicious packet drops and misroutings using header space analysis,” in *2016 8th International Symposium on Telecommunications (IST)*, pp. 521–526, IEEE, 2016.

- [11] A. Arsalan and R. A. Rehman, "Prevention of timing attack in software defined named data network with vanets," in *2018 International Conference on Frontiers of Information Technology (FIT)*, pp. 247–252, 2018.
- [12] T. A. Pascoal, Y. G. Dantas, I. E. Fonseca, and V. Nigam, "Slow team exhaustion ddos attack," in *IFIP International Conference on ICT Systems Security and Privacy Protection*, pp. 17–31, Springer, 2017.
- [13] X. Huang, K. Xue, Y. Xing, D. Hu, R. Li, and Q. Sun, "Fsdm: Fast recovery saturation attack detection and mitigation framework in sdn," in *2020 IEEE 17th International Conference on Mobile Ad Hoc and Sensor Systems (MASS)*, pp. 329–337, Dec 2020.
- [14] M. S. Farooq, S. Riaz, and A. Alvi, "Security and privacy issues in software-defined networking (sdn): A systematic literature review," *Electronics*, vol. 12, no. 14, 2023.
- [15] W. B. Jaballah, M. Conti, and C. Lal, "A survey on software-defined vanets: benefits, challenges, and future directions," *arXiv preprint arXiv:1904.04577*, 2019.
- [16] R. Abreu, F. Branco, M. J. C. S. Reis, and C. Serôdio, "Cybersecurity in connected and autonomous vehicles: A systematic review of automotive security," *IEEE Access*, vol. 13, pp. 116818–116855, 2025.
- [17] A. L. Aliyu, P. Bull, and A. Abdallah, "Performance implication and analysis of the openflow sdn protocol," in *2017 31st International Conference on Advanced Information Networking and Applications Workshops (WAINA)*, pp. 391–396, March 2017.
- [18] K. Bu, A. Laird, Y. Yang, L. Cheng, J. Luo, Y. Li, and K. Ren, "Unveiling the mystery of internet packet forwarding: A survey of network path validation," *ACM Computing Surveys (CSUR)*, vol. 53, no. 5, pp. 1–34, 2020.
- [19] M. Arif, G. Wang, O. Geman, V. E. Balas, P. Tao, A. Brezulianu, and J. Chen, "Sdn-based vanets, security attacks, applications, and challenges," *Applied Sciences*, vol. 10, no. 9, 2020.
- [20] S. Zhai, Y. Yang, J. Li, C. Qiu, and J. Zhao, "Research on the application of cryptography on the blockchain," *Journal of Physics: Conference Series*, vol. 1168, p. 032077, feb 2019.
- [21] U. Kaur, A. N. Mahajan, S. Kumar, and K. Dutta, "Security vulnerabilities in vanets and sdn-based vanets: A study of attacks," *International Journal of Computer Networks and Applications (IJCNA)*, vol. 11, no. 6, pp. 774–802, 2024.
- [22] M. Dibaei, X. Zheng, Y. Xia, X. Xu, A. Jolfaei, A. K. Bashir, U. Tariq, D. Yu, and A. V. Vasilakos, "Investigating the prospect of leveraging blockchain and machine learning to secure vehicular networks: A survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, pp. 683–700, Feb 2022.

- [23] M.-H. Hsu and C.-C. Liu, “A decentralized framework for the detection and prevention of distributed denial of service attacks using federated learning and blockchain technology,” *Engineering Proceedings*, vol. 92, no. 1, 2025.